Pointers

CSC 240 Data Structures Fall 2025

Table of Contents

1. Objectives

We don't assume any knowledge of pointers but if you do have any, it will be useful to review the concept and the practice of pointers.

1. Understand what a pointer is and how it differs from a reference.
2. Use pointers with arrays and functions.
3. Perform basic pointer arithmetic.
4. Dynamically allocate and deallocate memory using raw pointers.
5. Introduce the concept of smart pointers and avoid memory leaks.

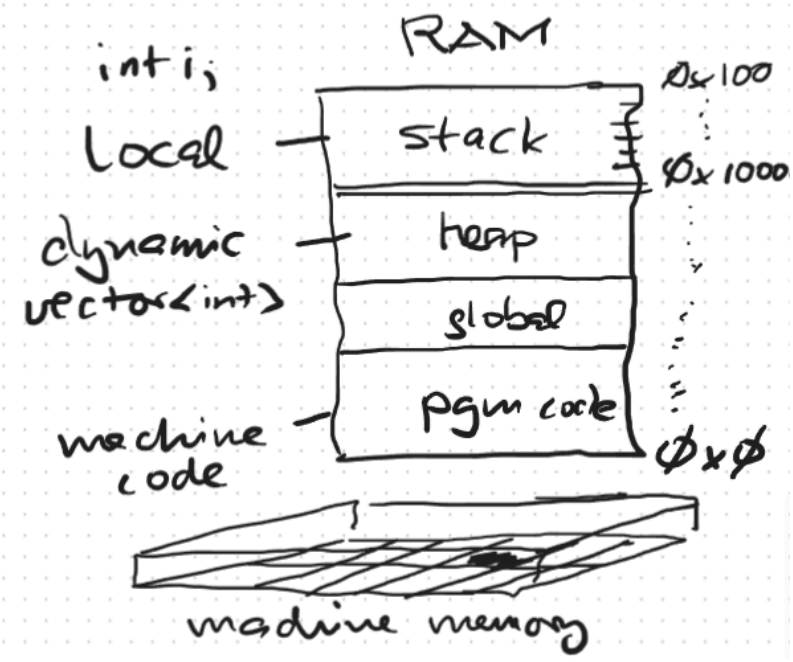
All topics:

- [ ] Getting the Address of a Variable
- [ ] Pointer and Reference Variables
- [ ] Relationship between Arrays and Pointers
- [ ] Pointer Arithmetic
- [ ] Initializing Pointers
- [ ] Pointers as Function Parameters
- [ ] Dynamic Memory Allocation

2. Getting the Address of a Variable

- The address-of operator `&` returns the memory address of a variable.
- Where exactly is this "memory address" on the computer?

    The memory address of most local variables is in the RAM (Random Access Memory) portion of the computer, somewhere in the **stack** segment. Dynamically allocated memory that can shrink or grow at runtime (via `new` or `malloc`) is in the **heap** segment.



- When the variable is undefined, you get any old value:

```
double amount; // double precision floating-point variable
cout << "Address: " << &amount // address-of variable
     << ", value = " << amount // value of variable
     << ", Size = " << sizeof(amount) << " bytes." << endl;
```

```
Address: 0x7fffd80e2470, value = 6.57115e-310, Size = 8 bytes.
```

- Why are the addresses always different when you re-run this code?

  Because of ASLR (Address Space Layout Randomization): Done for security reasons - much harder to inject code in changing memory.

- The following fragment declares three variables.

```
char letter;  // 1 byte   ( 8 bits)
short number; // 2 bytes  (16 bits)
float amount; // 4 bytes  (32 bits)
```

- In the computer memory, they may be arranged like this:

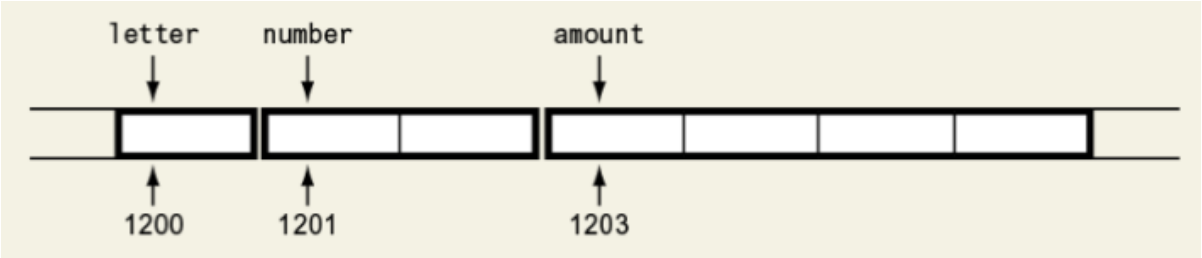


Figure 1: Variables and their addresses (Gaddis fig 9-1).

- The address itself is represented in hexadecimal (base 16) format. On a 64-bit computer system, the length of an address is typically 64 bits or 8 bytes.
- How many addresses can be represented on a 64-bit computer system?

  On a 64-bit computer system, every bit can have 2 values (0 or 1), which means that there are 2^{64} (18 quintillion) unique addresses.

  Incidentally, the reason why No Man's Sky can generate about 18 quintillion planets is because their universe uses a 64-bit generation seed leadeing to that many possible planet addresses.
- Is each byte in the computer's memory assigned to a unique address? If so, how would we know this?

  Yes, each byte in a computer's memory is assigned to a unique address.
- Demonstration:

```c
char a = 'A', b = 'B'; // two variables defined consecutively
printf("%p\n", (void*)&a); // using a 'generic' pointer `void*`
printf("%p\n", (void*)&b); // addresses 1 byte apart
```

```
0x7ffc31505896
0x7ffc31505897
```

- Why the generic pointers?

  `printf` expects `void*` that can hold the address of any object. This is required by the C-standard for `%p`. Without the cast, there is a type mismatch of `char*` (argument) vs. `void*` (expected by `printf`).

3. PRACTICE Using the address-of operator

- Write a C++ program that
  1. Declares an integer variable `x` and assigns it the value `25`.
  2. Prints the address of `x`, `&x`.
  3. Prints the size of `x` in bytes, `sizeof(x)`
  4. Prints the value stored in `x`.
- Sample output:

```
The address of x is 0x7ffc1970c684
The size of x is 4
The value stored in x is 25
```

- Solution:

```
// This program uses the & operator to determine the address of a
// variable and the sizeof operator to determine its size.

#include <iostream>
using namespace std;

int main()
{
  int x = 25;

  cout << "The address of x is "      << &x        << endl
       << "The size of x is "         << sizeof(x) << endl
       << "The value stored in x is " << x         << endl;
  return 0;
}
```

```
The address of x is 0x7ffcf8ff6c44
The size of x is 4
The value stored in x is 25
```

```
The address of x is 0x7ffe0929bf24
The size of x is 4
The value stored in x is 25
```

4. Pointer Variables

- A pointer variable is a variable that holds a memory address. When it is used to point at data, its type has to coincide with the type of the data: An integer pointer can point at an integer, etc.
- When an array is passed to a function, it "decays" to a pointer to the first array element.

- What happens in the following code example with regard to variable addresses?

```
#include <iostream>
using namespace std;

void showValues(int [], int);

int main()
{
  const int SIZE = 5;
  int numbers[SIZE] = {1,2,3,4,5};
  showValues(numbers,SIZE);
  return 0;
}

void showValues(int values[], int size)
{
  for (int count = 0; count < size; count++)
```

```
    cout << values[count] << " ";
}
```

```
1 2 3 4 5
```

- Explanation:

    1. In `main`, the name of the array `numbers` and its `SIZE` are passed to the function `showValues`.
    2. In the function, the `values` parameter receives the **address** of the array `numbers`. It points to `numbers[0]`.
    3. Inside the function, anything done to the `values` parameter is done to the `numbers` array.

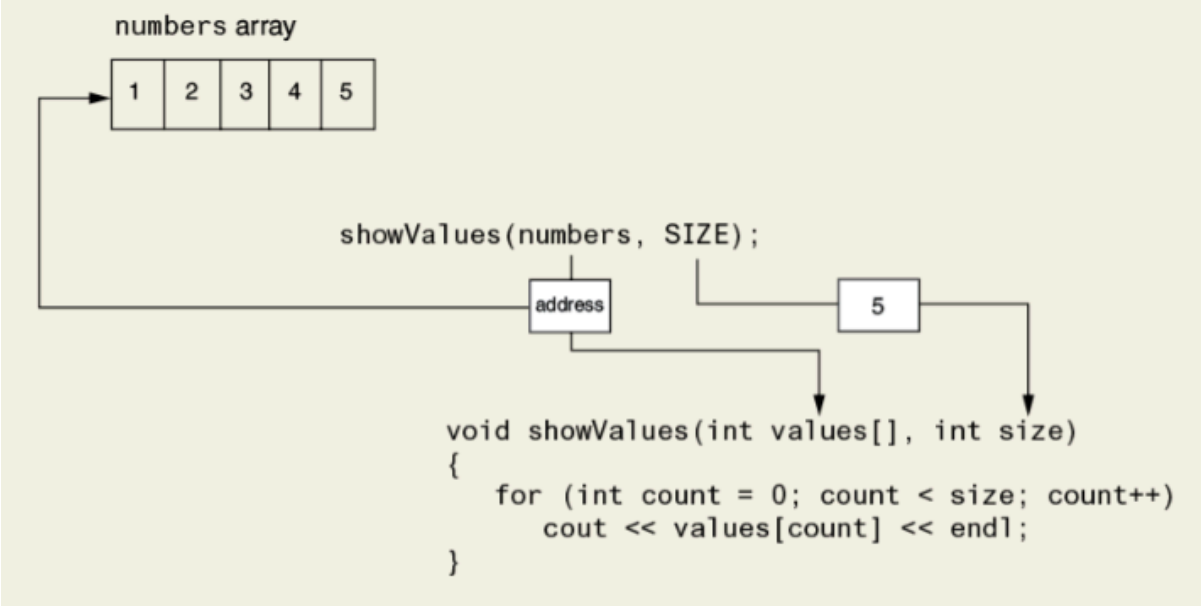


Figure 2: Passing an array to a function (fig. 9-2, Gaddis)

5. Reference variables

- A reference variable acts as an alias for another variable. Anything you do to the reference variable is done to the variable it references.

- What happens in the following code example with regard to variable references?

```
#include <iostream>
using namespace std;

void getOrder(int&);

int main()
{
  int jellyDonuts;
  getOrder(jellyDonuts);
```

```
  return 0;
}

void getOrder(int& donuts)
{
  cout << "How many donuts do you want? ";
  cin >> donuts;
  cout << "\nOkay, " << donuts << " it is.\n\n";
}
```

```
How many donuts do you want?
Okay, 4 it is.
```

- Explanation:

    1. In the function `getOrder`, the `donuts` parameter is a **reference** variable.
    2. The reference variable receives the **address** of the `jellyDonuts` variable (the argument of the `getOrder` function in `main`).
    3. `donuts` works like a pointer because it points to the `jellyDonuts` variable.

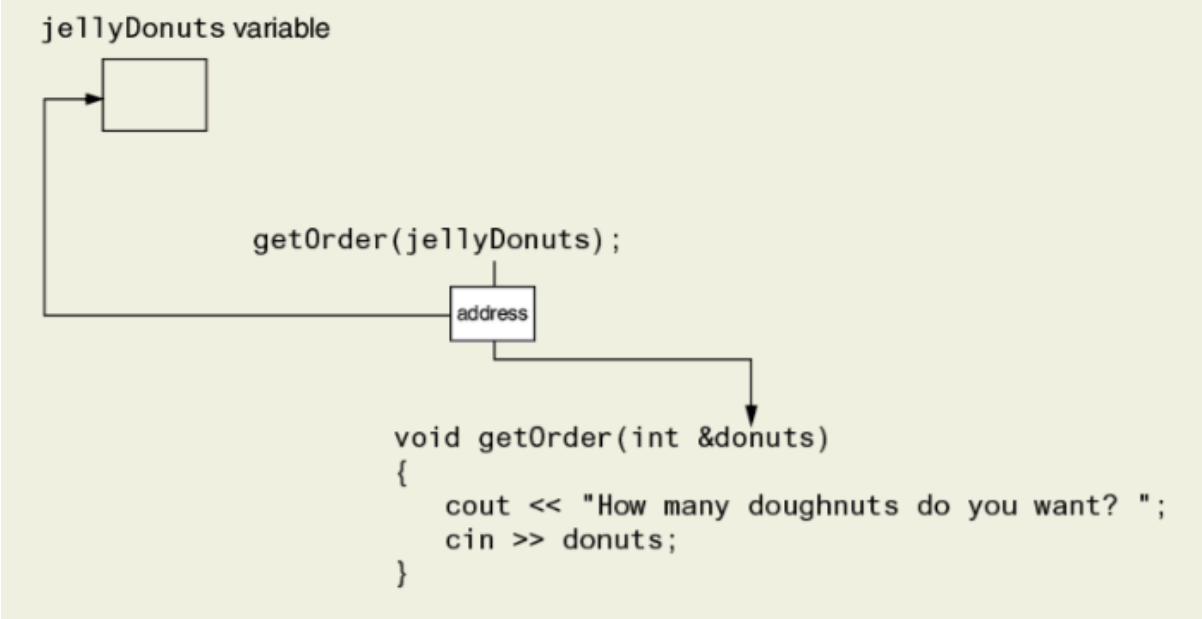


Figure 3: Passing an argument by reference (fig. 9-3, Gaddis)

- Compile and run the code above on the shell.

```
cd ../src
g++ donuts.cpp -o donuts
echo "4" | ./donuts
```

- Create input file `../data/donuts.txt`

```
cd ../data
echo "4" > donuts.txt
cat donuts.txt
```

```
4
```

6. Pointers vs. references and reason for pointers

- Pointer variables are similar to reference variables but they require more work from you.
- To make a pointer variable reference another item in memory, you have to write code to **fetch** the memory address of that item and **assign** it to the pointer variable (using the address-of operator &).
- When you use a pointer to **store** a value in the memory location that the pointer references, you have to specify that it should be stored at a location rather than in the pointer variable itself (using the indirection or dereference operator *).

- Programmers use reference variables to make changes permanent between calling routines and functions. But why use pointers?

    1. Pointers are necessary for **dynamic memory allocation**: whenever you need to work with an unknown amount of data.
    2. Pointers are useful in **algorithms** that manipulate arrays and especially strings (text).
    3. In object-oriented programming (OOP), pointers are useful for creating and working with objects and for **sharing access**.

- Example: Walking through a string using a pointer

```
#include <iostream>

int main() {
  const char* greeting = "Hello, world!"; // Declare a C-style string
                                          // as constant (literal)
                                          // character pointer.
  std::cout << greeting << std::endl;     // Print the whole string.

  // The pointer points at greeting[0].
  std::cout << "First letter: " << *greeting << std::endl;       // 'H'
  // Moving forward, the pointer points at greeting[1]
  std::cout << "Next letter: " << *(greeting + 1) << std::endl;  // 'e'

  // Traverse the C-style string with a pointer
  const char* p; // declare a character pointer p
  p = greeting; // assign &greeting[0] to p
  while (*p != '\0') {
    std::cout << *p;
    ++p;
  }
  std::cout << std::endl;
  return 0;
}
```

```
Hello, world!
First letter: H
Next letter: e
Hello, world!
```

7. Creating and using pointer variables

- What does `int* ptr;` mean?

    1. `ptr` is a pointer variable that points to an integer variable.
    2. `ptr` can hold the value of a computer memory address.
    3. Currently, `ptr` is unassigned (it can point towards any address).

- What's the difference between `int* ptr` and `int *ptr` ?

    There is no difference between `int* ptr` and `int *ptr`.

    `int* ptr` emphasizes that the data type of `ptr` is not `int` but pointer-to-int, and distinguishes the use of `*` from its use as indirection/dereference operator.

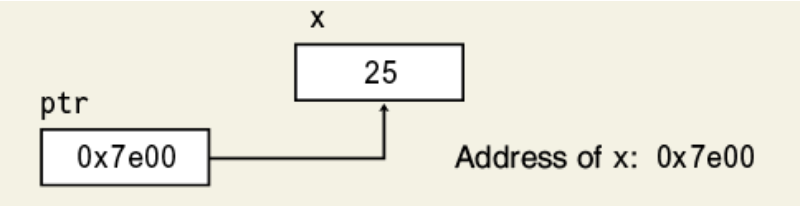
- **Rule:** Always initialize a pointer variable with the `nullptr` - otherwise you're changing a memory address somewhere.

```
int* foo = nullptr; // C++11
int* bar = NULL;    // C or older C++

printf("%p %p\n", foo, bar); // (nil) is NULL in Emacs Lisp
cout << foo << " " << bar;   // prints 0
```

```
(nil) (nil)
0 0
```

8. PRACTICE Store integer address in a pointer

- The following program defines an integer variable x, assigns a value to it, defines a pointer ptr and assigns the null pointer to it. Then the pointer is made to point at the variable address and both the value and the address are printed.



- **Task**: Open the starter code in OneCompiler.com, comment it where indicated, and complete the `cout` statements.

- Starter code:

```
// Store integer variable address in a pointer.
#include <iostream>
using namespace std;

int main()
```

```
{
  int x = 25;          // Assign value to an integer variable.
  int *ptr = nullptr; // Define pointer that points nowhere in particular
  ptr = &x;           // Store address of integer variable in pointer

  cout << "The value in x is "   << *prt  << endl;
  cout << "The address of x is " << ptr  << endl;

  return 0;
}
```

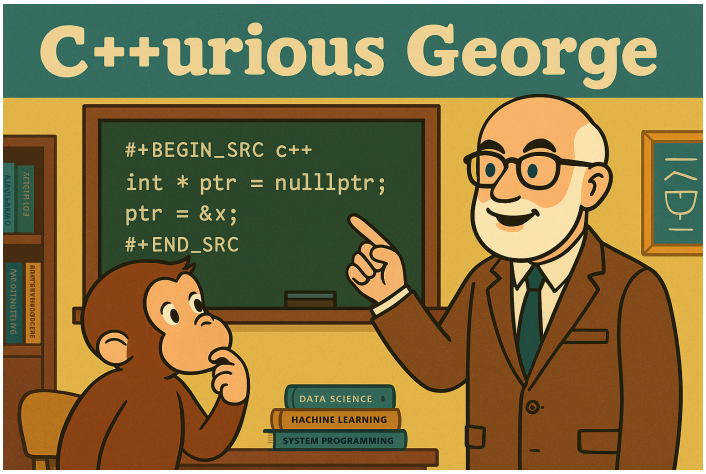
```
The value in x is 25
The address of x is 0x7fff60a94f4c
```

- Solution (working example with comments)

```
#include <iostream>
using namespace std;

int main()
{
  int x = 25;          // Assign value to integer variable.
  int *ptr = nullptr; // Pointer points nowhere in particular.
  ptr = &x;           // Store address of variable in pointer

  cout << "The value in x is "   << *ptr   << endl;
  cout << "The address of x is " << ptr << endl;

  return 0;
}
```

```
The value in x is 25
The address of x is 0x7ffd7656453c
```

9. C++urious George



- Can one assign a specific address directly to a pointer?

  Yes, one can, but it is dangerous and almost never recommended. You can assign a specific address to a pointer using a cast, like `int* p = (int*)0x7ffee4a2b4fc;`. However, if you access an invalid or restricted memory location, your program may crash or cause undefined behavior. Safe pointer use relies on assigning the address of a known variable or using dynamic memory allocation.

```
int* p = (int*)0x7ffee4a2b4fc;
cout << p; // *p will give segmentation fault
```

```
0x7ffee4a2b4fc
```

- What does the address `0x7e00` tell us about the memory layout of the program (e.g., stack location, variable storage, or memory limits), and how does it relate to the system's overall memory architecture?

  The address `0x7e00` shows that the variable or data it refers to is located at physical memory address 32,768 (in decimal). This is a relatively low address - you'd never see such a low address.

  However, the address alone does not reveal whether the system is 16-, 32-, or 64-bit—it simply indicates that this specific value fits within all of those architectures.

```
echo "ibase=16; 7E00;" | bc # 7 * 16^3 + 14 * 16^2 = 7 * 4096 + 14 * 256
```

```
32256
```

10. Indirect access to variables

- Pointers allow you to indirectly access and modify the variable being pointed to using the **indirection** operator *

- Example program:

  1. Assign the address of a variable to a pointer.
  2. Print the value of the variable using the dereferenced pointer.
  3. Assign a value to the location pointed to by the pointer.
  4. Print the value of the variable using the dereferenced pointer.

```
// This program demonstrates the use of the indirection operator.
#include <iostream>
using namespace std;

int main()
{
  int x = 42;         // Integer variable
  int *ptr = nullptr; // Pointer variable that can point to an int
  ptr = &x;           // Store the address of x in the pointer

  // Use x and ptr to display the VALUE in x
  cout << "Value of x (direct):               " << x    << endl
       << "Value of x (indirect via pointer): " << *ptr << endl;
```

```
  // Assign 99 to the location pointed to by ptr.
  // This will assign 99 to x.
  (*ptr) = 99;

  // Use x and ptr to display the VALUE in x
  cout << "Value of x (direct):                " << x    << endl
       << "Value of x (indirect via pointer): " << *ptr << endl;

  return 0;
}
```

```
Value of x (direct):                42
Value of x (indirect via pointer): 42
Value of x (direct):                99
Value of x (indirect via pointer): 99
```

- Explanation:

```
[ptr] ---> [x: 42]  // ptr is assigned &x

[ptr] ---*---> 42   // *ptr holds 42

[ptr] ---*---> 99   // *ptr is assigned 99

[ptr] ---> [x: 99]  // x holds 99
```

11. Re-pointing pointers

- Since pointers hold memory addresses, they can be re-pointed to other addresses. That happens when you initialize a pointer with the `nullptr` and then assign the address `&x` of an existing variable `x`.
- The following code uses one pointer `ptr` to change the values in three different variables:

```
// Define two integer variables and one integer pointer.
int x = 25, y = 50;
int *ptr = nullptr;

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;

// Use the pointer to manipulate the variables.

ptr = &x;       // Store address of x in pointer
                // Pointer points at x
(*ptr) += 100;  // Add 100 to the value in x

ptr = &y;       // Store address of y in pointer
                // Pointer points at y
(*ptr) += 100;  // Add 100 to the value in y

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;
```

```
x = 25, y = 50
```

```
x = 125, y = 150
```

12. PRACTICE Working with pointers

- Complete the starter code in: tinyurl.com/pointer-work
    1. Initialize the pointer with the `NULL` pointer.
    2. Assign the address of the variable to the pointer.
    3. Change the value of the variable using a pointer.
    4. Display the contents of the variable using a pointer.

- Sample output:

```
The number's value is 99.
```

- Starter code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
  // Assign 42 to the integer `number`.

  // Assign the null pointer to an integer pointer `ptr`.

  // Assign the address of number to the pointer.

  // Change value of number to 99 using the pointer.

  // Display the contents of the variable number using a pointer.

  return 0;
}
```

```
The number's value is 99.
```

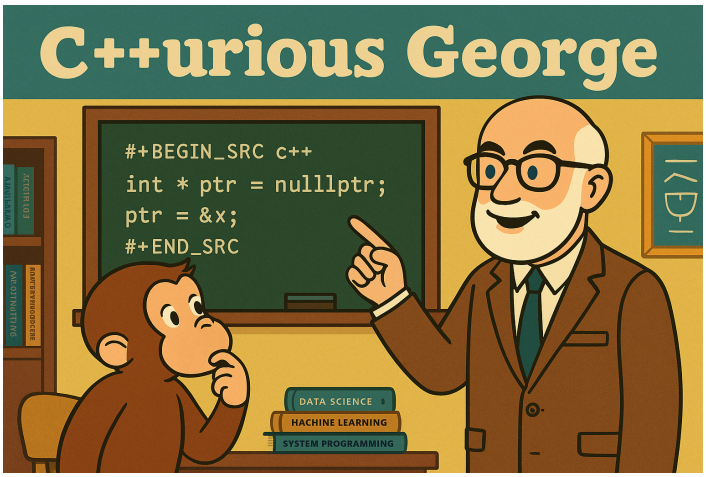
- Solution code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
  int number = 42; // Assign 42 to number.

  int *ptr = nullptr; // Assign null pointer to integer pointer.

  ptr = &number;   // Assign address of number to pointer.

  (*ptr) = 99;     // Change value of number to 99 using the pointer.

  // Display the contents of the variable number using a pointer.
  cout << "The number's value is " << *ptr << ".\n";
```

```
  return 0;
}
```

```
The number's value is 99.
```

13. C++urious George



- What happens if you assign the address of a non-existing variable to a pointer?

```
int *ptr = num; // num was not declared in this scope - compile error
```

- A pointer stores the value of an address to a variable. When it is re-pointed so that it stores the value of another variable, what happens to the address of the first variable?

  Nothing happens to the address of the first variable. It still exists in memory. Only the pointer has changed to store a new address.

  Unless there are other pointers pointing to the first variable, its address remains unused but intact. This is different from **dynamic memory**, where losing the only pointer to allocated memory causes a memory leak.

- Where exactly is the `nullptr` pointing at?

  `nullptr` is a special constant in C++ that represents a pointer that does not point to any valid memory location. Internally, it is represented as the address 0 (or a special null representation), but you should not assume it maps to a real memory region.

  Accessing or dereferencing a `nullptr` results in undefined behavior, typically a segmentation fault.

- What happens if you try to access or dereference a `nullptr`?

Accessing a `nullptr`'s value is undefined (may return zero). Dereferencing a `nullptr` results in undefined behavior, typically a segmentation fault.

```
int *ptr = nullptr;
//cout << *ptr;
```

- If `ptr` is a pointer variable that holds the value of an address, and `*ptr` is the value of that variable, what is `**ptr` and what is `&*ptr` and are these allowed? Test your assumptions.

```
int foo = 42; // integer variable
int *ptr = &foo; // pointer-to-int (stores address of int)
int **ptr2 = &ptr; // pointer-to-pointer-to-int (stores address of
                   // pointer-to-int)
cout << ptr << " " << *ptr << " " << &*ptr << endl;
cout << *ptr2 << " " << **ptr2;
```

```
0x7ffe7adbe394 42 0x7ffe7adbe394
0x7ffe7adbe394 42
```

---

14. PRACTICE Pointer challenge

1. Write a statement that defines a pointer variable named `fltPtr`. The variable should be a pointer to a `float`. Be sure to properly initialize the variable.

```
float *fltPtr = nullptr;
```

2. Define an integer variable `foo` and a pointer-to-int variable `ptr`. Write a statement that assigns the address of the `foo` variable to the `ptr` variable.

```
int foo;  // Does not have a defined value.
int *ptr=nullptr; // Points towards 0 or NULL.
ptr = &foo;  // Store address of foo in ptr.
```

3. Does `foo` have to have a value for this to work?

   No, `foo` does not have to have a value for this to work. `ptr` holds the value of the **address** of `foo`. The value `*ptr` is undefined.

4. Define `ptr` as a pointer variable that points to another variable `foo`. Write a `cout` statement that displays the value that `ptr` points to.

```
int foo;
int *ptr = &foo; // the asterisk indicates that ptr is a pointer variable
cout << *ptr; // the asterisk dereferences the pointer to access the
              // value of the variable the pointer points to
```

```
31340
```

5. List three uses of the * (asterisk) symbol in C++.

    1. Multiplication symbol.
    2. Pointer definition symbol.
    3. Indirection operator (dereferencing symbol).

6. What is the output of the following code?

```
int x=50, y=60, z=70;
int *ptr = nullptr;

cout << x << " " << y << " " << z << endl; // 50 60 70

ptr = &x;
(*ptr) *= 10; // 500
ptr = &y;
(*ptr) *= 5;  // 300
ptr = &z;
(*ptr) *= 2;  // 140

cout << x << " " << y << " " << z << endl; // 500 300 140
```

```
50 60 70
500 300 140
```

15. 9.3 Arrays and pointers

- What is the relationship between arrays and pointers?

    Array names can be used as constant pointers (that is you cannot change it to point it elsewhere), and pointers can be used as array names.

- Example: What will this program print?

```
short numbers[] = {10, 20, 30, 40, 50};
cout << *numbers << endl;
```

```
10
```

- How can one retrieve the entire array `numbers` using a pointer?

```
short numbers[] = {10, 20, 30, 40, 50};
cout << *numbers       << " "
<< *(numbers + 1) << " "
<< *(numbers + 2) << " "
<< *(numbers + 3) << " "
<< *(numbers + 4) << endl;
```

```
10 20 30 40 50
```

- Do you think this would work as a loop?

```
short numbers[] = {10, 20, 30, 40, 50};
for (int i=0; i<5; i++)
  cout << *(numbers + i) << " ";
```

```
10 20 30 40 50
```

- Would it work as a range-based loop, too?

```
short numbers[] = {10, 20, 30, 40, 50};
for (int num : numbers)
  cout << *(numbers + i) << " ";
```

  No it won't work because the range-based loop does not know subscripts (`i`).

- In fact there's an even more pointer-ly way to loop since the array name is a constant pointer (to the array's first element):

```
short number[] = {10, 20, 30, 40, 50};
for (short *p = number; p < number + 5; p++)
  cout << *p << " ";
```

```
10 20 30 40 50
```

- Can you understand how this `for` loop works?

  1. The **start** point(er) is `number[0]`, accessed as a deferenced pointer `*p`.
  2. The **stop** point(er) checks if the address of `p` is smaller than the address of `number[0]` offset by five `short` integers (2 byte).
  3. The **incrementor** moves the pointer by one address of 2 bytes each.

- How can you be sure that `short` is 2 bytes long?

```
short x;
cout << sizeof(x);
```

```
2
```

16. Why you can step through arrays using pointers

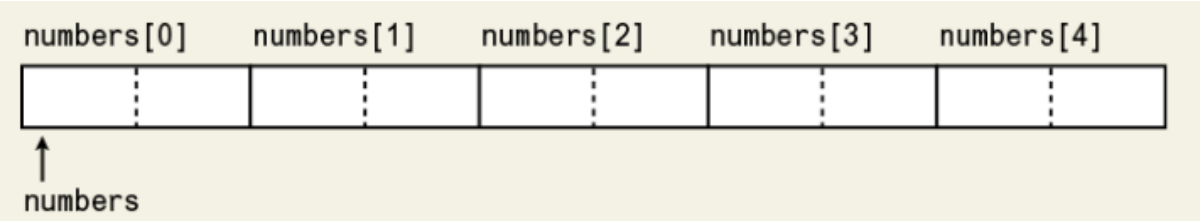


Figure 4: Array elements are stored in contiguous memory locations (Gaddis Fig. 9-5)

When you add a value to a pointer, you are adding that value times the size of the data type being referenced by the pointer - `*(number + 1)` moves 2 bytes forward when `number` is a `short` variable, and 4 bytes when it is an `int` variable: So `*(number +1)` is actually `*(number + 1 * sizeof(short))` etc.

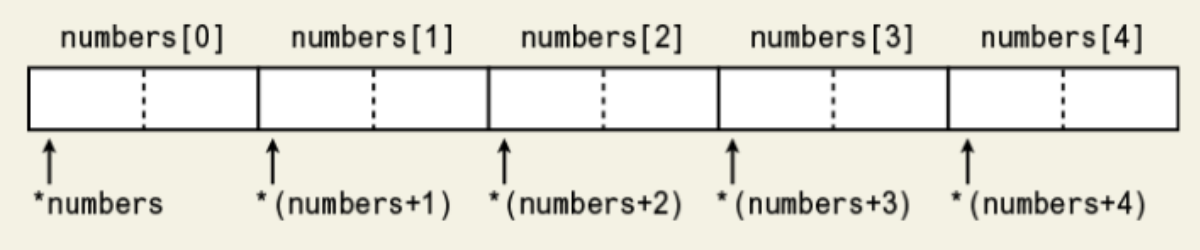


Figure 5: Subscript vs. pointer notation (Gaddis Fig. 9-6)

```
cout << "Size of short: " << sizeof(short) << " bytes.\n";
cout << "Size of short: " << sizeof(int)   << " bytes.";
```

```
Size of short: 2 bytes.
Size of short: 4 bytes.
```

But the whole operation works because moving like that through memory makes the pointer point at the next array element.

- Remember the rule: `array[index]` is equivalent to `(array + index)`.

17. PRACTICE Using subscripts with pointers (code along)

- Open the starter code to code along with me: tinyurl.com/arrays-pointers - Upload your final URL to Canvas.
- This program reveals the deep relationship between arrays and pointers once more. We can come up with four different ways of printing the array elements using pointers.

- In the following program, subscript notation is used with a pointer variable, and pointer notation is used with an array name.

    1. An array of `double coins` is created.
    2. A pointer-to-double `doublePtr` is created.
    3. `coins` is assigned to `doublePtr` (no `&` operator needed!)
    4. The `coins` array is printed using the subscripted `doublePtr`.
    5. The `coins` array is printed using pointer notation.
    6. The `coins` array is printed using pointer arithmetic.
    7. The `coins` array is printed using referencing and dereferencing.

```
// Demonstrate connection between arrays and pointers
// Create an array of double variables, and a pointer-to-double
```

```
// Display the array elements using pointers in four ways.

const int NUM_COINS = 5;
double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
double *doublePtr = nullptr;

// Point the pointer at the array.
doublePtr = coins;

// Display array using a pointer subscription.
for (int count = 0; count < NUM_COINS; count++)
  cout << doublePtr[count] << " ";

cout << endl;

// Display array using pointer notation.
for (int count = 0; count < NUM_COINS; count++)
  cout << *(coins + count) << " ";

cout << endl;

// Display array using pointer arithmetic.
for (double *p = coins; p < coins + NUM_COINS; p++)
  cout << *p << " ";

cout << endl;

// Display array with referenced array and dereferenced pointers
for (int count = 0; count < NUM_COINS; count++)
  {
    doublePtr = &coins[count]; // store element in pointer
    cout << *doublePtr << " "; // print dereferenced pointer
  }
```

```
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
```

18. PRACTICE Stepping through an array with pointers (homework)

- Write a program that simulates a countdown sequence for a rocket launch. Complete and run the starter code (below) twice:
    1. Step through the array using pointer notation.
    2. Step through the array using pointer subscription.
    3. Step through the array using pointer arithmetic.

- Sample output:

```
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
```

- Starter code:

```
// This program simulates a countdown using an array and pointers.
// The address of the array is assigned to a pointer variable.  The
```

```
// array is printed using first pointer notation, and then subscripted
// pointers.

#include <iostream>
using namespace std;

int main()
{
  const int SIZE = 5;
  int numbers[SIZE] = {5,4,3,2,1};
  int index;
  // Initialize the numbersPtr pointer.


  // Assign the address of the `numbers` array to the `numbersPtr`
  // pointer variable.


  cout << "Countdown start! ";

  // Solution 1) pointer notation:
  // Step through the array `numbers` using the loop variable `index`.

  // Print the array elements using pointer notation.


  cout << " Blastoff!" << endl;
  cout << "Countdown start! ";

  // Solution 2) pointer subscription:
  // Step through the array `numbers` using the loop variable `index`.

  // Print the array elements using pointer notation.


  cout << " Blastoff!" << endl;
  cout << "Countdown start! ";

  // Solution 3) pointer arithmetic
  // Step through the array `numbers` using a pointer loop variable

  // Print the array elements using pointer arithmetic


  cout << " Blastoff!" << endl;

  return 0;
}
```

```
Countdown start!  Blastoff!
Countdown start!  Blastoff!
Countdown start!  Blastoff!
```

- Solution:

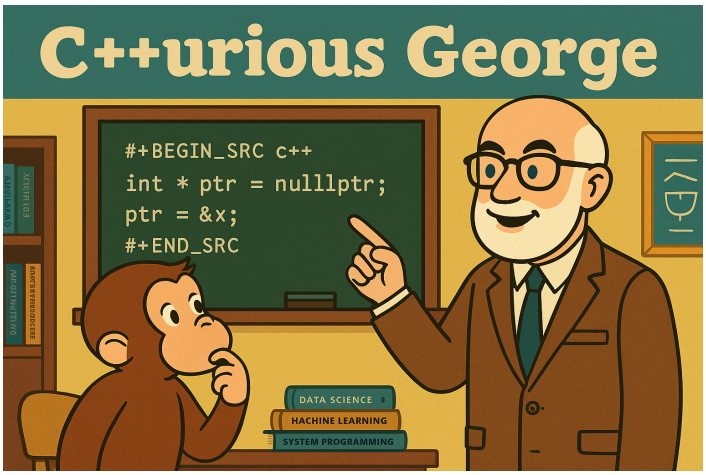
```
// This program simulates a countdown using an array and pointers.
// The address of the array is assigned to a pointer variable.  The
// array is printed using first pointer notation, and then subscripted
// pointers.

#include <iostream>
```

```
using namespace std;

int main()
{
  const int SIZE = 5;
  int numbers[SIZE] = {5,4,3,2,1};
  int index;
  // Initialize the numbersPtr pointer.
  int *numbersPtr = nullptr;

  // Assign the address of the `numbers` array to the `numbersPtr`
  // pointer variable.
  numbersPtr = numbers;

  cout << "Countdown start! ";

  // Solution 1) pointer notation:
  // Step through the array `numbers` using the loop variable `index`.
  for (index = 0; index < SIZE; index++)
    // Print the array elements using pointer notation.
    cout << *(numbers + index) << "...";

  cout << " Blastoff!" << endl;
  cout << "Countdown start! ";

  // Solution 2) pointer subscription:
  // Step through the array `numbers` using the loop variable `index`.
  for (index = 0; index < SIZE; index++)
    // Print the array elements using pointer notation.
    cout << numbersPtr[index] << "...";

  cout << " Blastoff!" << endl;
  cout << "Countdown start! ";

  // Solution 3) pointer arithmetic
  // Step through the array `numbers` using a pointer loop variable
  for (int *p = numbers; p < numbers + SIZE; p++)
    // Print the array elements using pointer arithmetic
    cout << *p << "...";

  cout << " Blastoff!" << endl;

  return 0;
}
```

```
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
```


```
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
Countdown start! 5...4...3...2...1... Blastoff!
```


19. C++urious George



- So what if you step through an array with a pointer and give the pointer an address outside of the array?

  C++ performs no out-of-bounds checking with arrays. So there will be no complaint but it's a disaster.

```
int num[3] = {1,2,3};
int *ptr;
for (ptr = num; ptr < num + 4; ptr++)
  cout << *ptr << " "; // last element out-of-bounds
```

```
1 2 3 -1018356224
```

- Why does this work: setting a pointer-to-double to the array name, and then printing the array using the pointer?

  This works because the array name can be used as a constant pointer, and the pointer can be used as an array name.

- Here's a cheat sheet for arrays & pointers: Can you set it to prose?

```
1. ptr[i] \equiv *(ptr + i)

2. array[i] \equiv *(array + i)

3. for (p = array; p < array + N; p++)

4. ptr = &array[i]; *ptr = array[i]
```

  In Prose:

  1. Pointer subscription equals dereferenced pointer arithmetic.
  2. Array subscription equals dereferenced array arithmetic.
  3. You can use a pointer as a loop counter.
  4. You can store array element addresses in pointers, and get their value from the dereferenced pointer.

- What is the difference between array names and pointer variables?

    The only difference between an array name and a pointer variable is that you cannot change the address an array name points to. Try it!

```
int foo[20], bar[20];
int *ptr = nullptr;

ptr = foo; // legal
ptr = bar; // legal

foo = bar; // illegal - incompatible types
bar = ptr; // illegal - incompatible types
```

20. Review questions

1. Can you dereference an array name with the indirection operator? As in this example: `int arr[10]; cout << *arr;`

    Yes, you can dereference an array name with * because the array name is equivalent to a (constant) pointer to its first element.

```
int arr[10] = {5,4,3,2,1};
cout << *arr; // Same as arr[0] because an array name is equivalent to
// a pointer to arr[0]
//int *ptr = arr;
//cout << *ptr; // points at arr[0]
//cout << *ptr[1]; // not allowed - must be *(ptr + 1);
//cout < *(arr +1); // not allowed - same as *(arr[0]+1) but arr[0] is
// not a pointer
```

```
5
```

2. Look at the following array definition:

```
int scores[5] = {454, 556, 445, 234, 343};
```

    Which of these `cout` statements would display the 5th element in the array `scores`? Are any of them illegal statements?

    - [ ] `cout << *(scores + 4);`
    - [ ] `cout << *(scores + 5);`
    - [ ] `cout << *scores + 4;`
    - [ ] `cout << *scores + 5;`
    - [ ] `cout << &scores[4];`
    - [ ] `cout << *scores[4];`

```
int scores[5] = {454, 556, 445, 234, 343};
cout << *(scores + 4) << endl; // correct
cout << *(scores + 5) << endl;
cout << *scores + 4   << endl;
cout << *scores + 5   << endl;
```

```
cout << &scores[4]    << endl;
//cout << *scores[4]    << endl; // illegal
```

```
343
30972
458
459
0x7ffc930bfc80
```

3. Is this true? Array names are pointer constants. You can't make them point to anything but the array they represent.

   > Yes, this is true! Array names decay to pointers to their first element but they are not true pointers: So while `int a[10]; int *p = a;` is valid, `a = p` is illegal. An array's address and size are fixed at compile time. This ensures the contiguous memory layout - unlike for example linked list data structures where elements are scattered and only connected via pointers.

4. Rewrite the following loop so that it uses pointer notation (with the indirection operator) instead of subscript notation.

   ```
   for (int x=0; x < 100; x++)
     cout << arr[x];
   ```

   Solution:

   ```
   int arr[100] {0};
   for (int x=0; x < 100; x++)
     cout << arr[x];

   arr[0]=1; cout << endl;

   for (int x=0; x < 100; x++)
     cout << *(arr + x);
   ```

   ```
   0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
   1000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
   ```

21. 9.4 Pointer arithmetic

- You can change pointer content with addition or subtraction. The following program demonstrates this. You have already seen pointera arithmetic in pointer-controlled array loops.

```
#include <iostream>
using namespace std;

int main()
{
  const int SIZE = 8;
  int set[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
  int *numPtr = nullptr;  // initialized pointer
  int count;              // loop counter
```

```
  // Make numPtr point to the set array.
  numPtr = set;

  // Use the pointer to display the array contents.
  cout << "The numbers in set are:\n:";
  for (count = 0; count < SIZE; count++)
    {
      cout << *numPtr << " "; // print array value pointed at
      numPtr++;               // move pointer to next array element
    }

  // Display array contents in reverse order.
  cout << "\nThe numbers in set backwards are:\n:";
  for (count = 0; count < SIZE; count++)
    {
      numPtr--;               // move pointer to previous array element
      cout << *numPtr << " "; // print array value pointed at
    }
  return 0;
}
```

```
The numbers in set are:
:5 10 15 20 25 30 35 40
The numbers in set backwards are:
:40 35 30 25 20 15 10 5
```

- Not all operations are allowed on pointers: `++` and `--`, `+=`, `-=`, are OK, and a pointer may be subtracted from another pointer.

Operation	Example
	`int vals[]={4,7,11};`
	`int *valptr = vals;`

`++, --`	`valptr++; // points at 7`
	`valptr--; // now points at 4`

`+, -` (pointer and `int`)	`cout << *(valptr + 2); // 11`

`+=, -=` (pointer and `int`)	`valptr = vals; // points at 4`
	`valptr += 2;  // points at 11`

`-` (pointer from pointer)	`cout << valptr-val; // difference`
	`//(number of ints) between valptr`
	`// and val`

22. PRACTICE Subtract a pointer from another pointer [opt]

- You can group pointer variables in arrays just like other variables, and initialize it with a `nullptr`:

```
int *ptr[10] = {nullptr};
```

- Write a program that demonstrates pointer arithmetic by substracting one pointer from another and printing the result.
    1. Define an array of two double-precision elements.
    2. Define a pointer-to-double array of two pointer variables.
    3. Store the array element addresses in the pointer array elements.
    4. Display the content of the pointer array elements.
    5. Check if the result depends on the data type by running the program for `bool` and `char` variables and pointers.

- Starter code:

```
// Demonstrate pointer arithmetic by substracting one pointer from
// another and test the dependency of the result on the data type.
#include <iostream>
using namespace std;

int main()
{
  // Define a double precision array of two elements.

  // Define a pointer-to-double array of two pointer variables.

  // Store the array element addresses in the pointer array elements.

  // Display the content of the pointer array elements.

  // Display the result of subtracting the second from the first
  // pointer array element.

  return 0;
}
```

- Sample output: (did you expect this result?)

```
ptr[0] = 0x7ffc3978afc0
ptr[1] = 0x7ffc3978afc8
ptr[1]-ptr[0] = 1
```

- Solution:

```
// Demonstrate pointer arithmetic by substracting one pointer from
// another and test the dependency of the result on the data type.
#include <iostream>
using namespace std;

int main()
{
  // Define a double precision array of two elements.
  double arr[2];
  // Define a pointer-to-double array of two pointer variables.
  double *ptr[2] = {nullptr};

  // Store the array element addresses in the pointer array elements.
  ptr[0] = &arr[0];
  ptr[1] = &arr[1];

  // Display the content of the pointer array elements.
  cout << "ptr[0] = " << ptr[0] << endl
```

```
        << "ptr[1] = " << ptr[1] << endl;

  // Display the result of subtracting the second from the first
  // pointer array element.
  cout << "ptr[1]-ptr[0] = " << ptr[1]-ptr[0] << endl;

  return 0;
}
```

```
ptr[0] = 0x7fffdc4d4850
ptr[1] = 0x7fffdc4d4858
ptr[1]-ptr[0] = 1
```

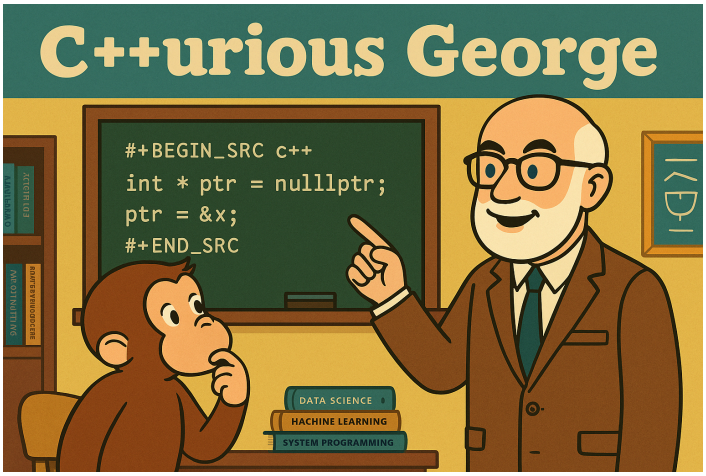
- Checking for other data types: All other data types show the same subtraction result. The `char` version shows an anomaly because the pointer values (addresses) are not printed. See here for a solution, and below ("C++urious George") for an explanation.

- Initializing an array of pointers and printing the values:

```
int *ptr[10] = {nullptr};
for (int i=0;i<10;i++) cout << *ptr << " "; // or `ptr` for addresses
```

```
0 0 0 0 0 0 0 0 0 0
```

23. C++urious George



- Will `ptr` print the same address than `&ptr`? Check it out.

```
int *ptr = nullptr;
cout << "nullptr: " << ptr << "\nAddress-of nullptr " << &ptr;
```

```
nullptr: 0
```

```
Address-of nullptr 0x7ffee7144830
```

- What happens if you subtract a pointer from the `nullptr`?

```
int *ptr = nullptr;
int x = 1;
int *xptr = &x;
printf("%p\n", (void*)(ptr - xptr));
```

```
0xffffe000584cc73f
```

- If `px` and `py` are pointer-to-int variables, what data type is `py-px`?

  The difference of two pointers has its own data type, `ptrdiff_t`. `std::ptrdiff_t` is a signed integer type. So for example two pointers pointing to

- When subtracting two pointer-to-char variables stored in a pointer array, using `cout` did not print anything - why? Can you fix it?

```
char *ptr = nullptr;
char x; ptr = &x;
cout << ptr << endl;  // prints nothing
printf("%p\n", ptr);
```

```
0x7ffd49e5f67f
```

  `char *` has a special meaning - it's a C-style string. Only `int *` and `double *` etc. print the pointer value (address) as expected. The problem is that `cout` tries to dereference and print characters until it finds a terminating `\0` character. If that's missing, nothing or garbage is printed.

  The solution is to cast to `void *` to let `cout` know "this is just a pointer": `cout << (void *)ptr;` Here, `void *` is the generic pointer type in C/C++.

24. Pointer arithmetic: Review questions

1. What will the output of this code be?

```
const int SIZE = 4;
int results[SIZE] = { 54, 44, 34, 22 };
int *resultsPtr = nullptr;
resultsPtr = results + 2;
cout << *resultsPtr << endl;
```

```
34
```

```
const int SIZE = 4;
int results[SIZE] = { 54, 44, 34, 22 };
```

```
int *resultsPtr = nullptr;
resultsPtr = results + 2; // move to results[0+2] = results[2]
cout << *resultsPtr << endl;  // 34
```

```
34
```

2. Can you multiply or divide a pointer?

   You cannot multiply or divide a pointer.

3. Assume `ptr` is a pointer to an `int` and holds the address 12000. On a system with 4-byte integers, what address will be in `ptr` after the following statement?

```
ptr += 10;
```

   After the statement, the pointer will point to a place 10 * 4 bytes = 40 bytes forward in contiguous memory, and the new address will be 12000 + 40 = 12040.

4. Assume `pint` is a pointer variable. Is each of the following statements valid or invalid? If any is invalid, why?

```
A) pint++;
B) --pint;
C) pint /= 2;
D) pint *= 4;
E) pint += x; // x is an int
```

```
int x;
int *pint;
pint++;
pint += x;
// pint /= 2;  // not valid
// pint *= 4;  // not valid
```

25. 9.5 Initializing pointers

- You can initialize pointers at definition time:

```
int num, *numPtr = &num;
int val[3], *valPtr = val;
```

- When initializing pointers, you cannot mix data types:

```
double cost;
int *ptr = &cost; // ILLEGAL conversion of double* to int*
```

- You can test if a pointer is properly initialized or not.

```
int *ptr = nullptr; // properly initialized

if (!ptr)
  cout << "Pointer is null." << endl;
 else
   cout << "Pointer is non-null." << endl;

int *ptr2; // dangling pointer - not properly initialized
if (!ptr2)
  cout << "Pointer is null." << endl;
 else
   cout << "Pointer is non-null." << endl;
```

```
Pointer is null.
Pointer is non-null.
```

26. Initializing pointers: Review questions

1. In the following code, is the pointer variable definition legal or illegal?

```
int seats[5] = {2,4,5,7,6};
int *seats = seats;
```

   It's illegal since an array name (`seats`) stands for a constant pointer by the same name so this counts as a double declaration, and if you took the `int` away, `*seats = seats` is a conflict because the compiler cannot convert `int*` into `int`. The way to fix it is to give the pointer variable a different name: `seatsPtr`.

2. In the following code, is the assignment statement in the 2nd line legal or illegal?

```
double dollars;
int dollarsPtr = &dollars;
```

   It's illegal since the right hand value is an address (of the `double` variable `dollars`), which expects to be stored in a pointer-to-double of the data type `double*`.

3. Is each of the following definitions valid or invalid? If any is invalid, why?

```
A)  int ivar;
    int *iptr = &ivar;

B)  int ivar, *iptr = &ivar;

C)  float fvar;
    int *iptr = &fvar;

D)  int nums[50], *iptr = nums;

E)  int *iptr = &ivar;
    int ivar;
```

```
int ivar;            // integer variable
int *iptr = &ivar;   // address stored in pointer-to-int
```

```
int ivar2, *iptr2 = &ivar; // integer variable, pointer-to-int stores
// address

float fvar;          // floating-point variable
//int *iptr3 = &fvar;  // ILLEGAL: pointer-to-int cannot store
// floating-point variable address

int nums[50], *iptr4 = nums; // pointer-to-int stores address of
// nums[0], nums is constant pointer

int *iptr5 = &ivar;  // ILLEGAL - ivar5 not defined yet - will
// compile!
int ivar5;
```

27. 9.6 Comparing pointerse [opt]

- If one address comes before another address in memory, the first address is considered to be "less than" the second. Therefore, pointers can be compared using any relational operators.

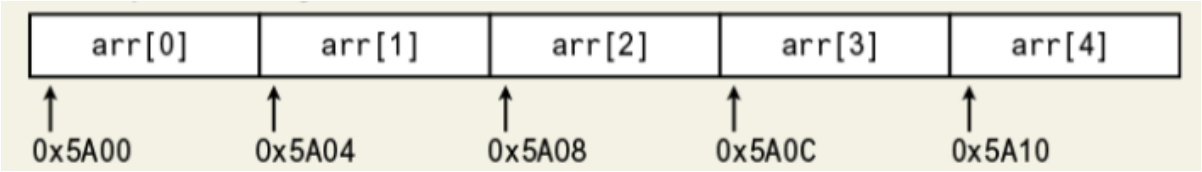


Figure 6: Array elements and array addresses (4 byte apart)

- Which relational operators are there?

```
>    greater
<    smaller
==   equal
!=   not equal
>=   greater or equal
<=   smaller or equal
```

28. Program: Using pointer comparison to avoid out-of-bounds errors [opt]

- The program below performs pointer arithmetic as seen before.

```
#include <iostream>
using namespace std;

int main()
{
  const int SIZE = 8;
  int numbers[SIZE] = {5,10,15,20,25,30,35,40};
  int *ptr = numbers; // ptr points to numbers

  // Display the numbers in the array.
```

```
  cout << "The numbers are:\n";
  cout << *ptr << " "; // Display first element only
  while (ptr < &numbers[SIZE-1]) // check if end is reached
    {
      // Advance ptr to point to next element.
      ptr++;
      // Display the value pointed to by ptr.
      cout << *ptr << " ";
    }

  // Display the numbers in reverse order.
  cout << "\nThe numbers backward are:\n";
  cout << *ptr << " "; // Display last element
  while (ptr > numbers) // check if beginning is reached
    {
      // Move ptr backward to previous element.
      ptr--;
      // Display the value point to by ptr.
      cout << *ptr << " ";
    }
  return 0;
}
```

```
The numbers are:
5 10 15 20 25 30 35 40
The numbers backward are:
40 35 30 25 20 15 10 5
```

29. Comparing pointers: Review questions [opt]

1. Are the following expressions `true` or `false`? Why?

```
int arr[5];

if (&arr[1] > &arr[0])  cout << "True.\n";
if (arr == &arr[0])     cout << "True.\n";
if (arr < &arr[4])      cout << "True.\n";
if (&arr[2] != &arr[3]) cout << "True.\n";
```

```
True.
True.
True.
True.
```

   Explanation:

```
if (&arr[1] > &arr[0])  // address of arr[1] comes after arr[0]
if (arr == &arr[0])     // arr is an alias for &arr[0]
if (arr < &arr[4])      // arr = &arr[0] comes before &arr[4]
if (&arr[2] != &arr[3]) // no two addresses can be identical
```

2. If `ptr1` and `ptr2` are two pointer variables, what's the difference between these two statements?

```
if (ptr1 < ptr2)
```

```
if (*ptr1 < *ptr2)
```

Explanation:

```
if (ptr1 < ptr2)    // checks if the address ptr1 points to comes
                    // before the address that ptr2 points to.
if (*ptr1 < *ptr2)  // checks if the value of the variable that
                    // ptr1 points to is smaller than the value of the
                    // variable that ptr2 points to.
```

3. Assume `numbers` is an array. Is the following expression `true` or `false`?

```
numbers > &numbers[1]
```

Check:

```
int numbers[10];
cout << boolalpha << (numbers > &numbers[1]);
```

```
false
```

4. Assume `numbers` is an array. Is the following expression `true` or `false`?

```
numbers == &numbers[0];
```

```
int numbers[10];
cout << boolalpha << (numbers == &numbers[0]);
```

```
true
```

5. Comparing two pointers is the same as comparing the values the two pointers point to. True or false - why?

> False - comparing two pointer variables means comparing the two addresses of the variables that the pointers point to. The pointers comparison with == is only `true` if both pointers point to the same address (of one and the same variable), in which case the value comparison is also `true`.

6. Assuming `arr` is an array of integers, will each of the following code snippets display `True` or `False`?[1]

```
A) (arr < &arr[1]) ? (cout << "True") : (cout << "False");

B) (&arr[4] < &arr[1]) ? (cout << "True") : (cout << "False");

C) (arr != &arr[2]) ? (cout << "True") : (cout << "False");

D) (arr != &arr[0]) ? (cout << "True") : (cout << "False");
```

```
int arr[10];

(arr < &arr[1]) ? (cout << "True\n") : (cout << "False\n");

(&arr[4] < &arr[1]) ? (cout << "True\n") : (cout << "False\n");

(arr != &arr[2]) ? (cout << "True\n") : (cout << "False\n");

(arr != &arr[0]) ? (cout << "True\n") : (cout << "False\n");
```

```
True
False
True
False
```

30. 9.7 Pointers as function parameters

- Pointers can be used as function parameters. This gives the function access to the original argument in the calling function, so it's pass-by-reference (address) rather than pass-by-value (copy).
- Short example:

```
#include <iostream>
using namespace std;

void increment(int *p) {
  (*p)++; // modify the value pointed to
}

int main() {
  int x = 10;
  increment(&x); // pass address of x
  cout << x; // prints 11
}
```

- This is also what using a reference parameter does. What's the difference?

    1. References are aliases for existing variables. No new storage is allocated and the original name can be used in the function.
    2. Pointers require explicit dereferencing (`*ptr`) to access or modify the pointed-to value.
    3. References cannot be reseated to another variable after initialization, unlike pointers.

- **Example:** This program uses two `void` functions that accept addresses of variables as arguments. `getNumber` asks the user for a number as input, and `doubleValue` doubles the input value. Neither function needs a `return` value because of the pass-by-reference.

```
#include <iostream>
using namespace std;

// Function prototypes
void getNumber(int *);
void doubleValue(int *);

int main()
{
```

```
  int number;

  // Call `getNumber` and pass the address of `number`.
  getNumber(&number);

  // Call `doubleValue` and pass the address of `number`.
  doubleValue(&number);

  // Display the value in `number`.
  cout << "That value doubled is: " << number << endl;

  return 0;
}

// Definition of getNumber. The parameter `input` is a pointer. The
// function asks the user for a number. The value entered is stored in
// the variable pointe to by `input`. The calling function's argument
// must be an address.
void getNumber(int *input)
{
  cout <<  "Enter an integer number: ";
  cin >> *input;
}

// Definition of doubleValue. The paramter `value` is a pointer. The
// function multiplies the variable pointed to by `value` by two. The
// calling function's argument must be an address.
void doubleValue(int* value)
{
  (*value) *= 2;
}
```

```
Enter an integer number: That value doubled is: 57492
```

- Test the program on the command line after tangling the source code.

```
$ make ptrpar
g++     ptrpar.cpp   -o ptrpar
$ ./ptrpar
Enter an integer number: 22
That value doubled is: 44
```

- What would happen if we would not use `*input` in the `cin` statement but instead `input`?

  If you write `cin >> input;` instead of `cin >> *input;`, you are telling `cin` to store the user input as an address in the pointer variable itself, not in the location it points to. This leads to a type mismatch: `cin` expects an `int&` (a reference to an int), but input is an `int*`.

31. Using pointer parameters to accept array addresses

- The following two (call-by-value) function prototypes are identical:

```
void function(int arr[], int size);
void function(int  *arr, int size);
```

- Example: Two functions to get sales data and add them up to total sales.

```
#include <iostream>
#include <iomanip>
using namespace std;

// Function prototypes
void getSales(double *, int);
double totalSales(double *, int);

int main()
{
  // Declare the sales data array.
  const int QTRS = 4;
  double sales[QTRS];

  // Set the numeric output formatting.
  cout << fixed << showpoint << setprecision(2);

  // Get the sales data for all quarters.
  getSales(sales, QTRS);

  // Display the totals sales for the year.
  cout << "\nThe total sales for the year are $";
  cout << totalSales(sales, QTRS) << endl;

  return 0;
}

// Definition of getSales. This function uses a pointer to accept the
// address of an array of doubles. The function asks the user to enter
// sales figures and stores them in the array.
void getSales(double *arr, int size)
{
  for (int count = 0; count < size; count++)
    {
      cout << "\nEnter the sales figure for quarter ";
      cout << (count + 1) << ": "; // pointer notation
      cin >> arr[count];
      cout << arr[count];
    }
}

// Definition of totalSales. This function uses a pointer to accept
// the address of an array. The function returns the total of the
// elements in the array.
double totalSales(double *arr, int size)
{
  double sum = 0.0;

  for (int count = 0; count < size; count++)
    {
      sum += *arr;
      arr++;       // pointer arithmetic
    }
  return sum;
}
```

- Input data:

```
cd ../data
echo "10263.98 12369.69 11542.13 14792.06" > sales.txt
cat sales.txt
```

```
10263.98 12369.69 11542.13 14792.06
```

32. PRACTICE Pointers as function parameters

- Write a C++ program with a function `increment` that has a pointer-to-integer parameter `ptr`:
  1. Define an integer `value` with the value `9`.
  2. Display `value`.
  3. Increment `value` using the function `increment`.
  4. Display the value.

- Sample output:

```
9
10
```

- Starter code: https://tinyurl.com/pointer-parameter-practice

```
// This program increments a set integer value using a function with
// a pointer-to-integer parameter.
#include <iostream>
using namespace std;

// Function prototype

int main()
{
  // Define an integer `value` with the value 9.

  // Display `value`.

  // Increment `value` using the function `increment`.

  // Display the value.

  return 0;
}

// Function definition
```

- Solution:

```
// This program increments a set integer value using a function with
// a pointer-to-integer parameter.
#include <iostream>
using namespace std;

// Function prototype
void increment(int *);
```

```
int main()
{
  // Define an integer `value` with the value 9.
  int value = 9;

  // Display the value.
  cout << value << endl;

  // Increment `value` using the function `increment`.
  increment(&value);

  // Display the value.
  cout << value << endl;

  return 0;
}

// Definition of increment. This function uses a pointer to accept an
// integer value. The function increments the value.
void increment(int *ptr)
{
  (*ptr)++; // (*ptr++); will not work - why?
}
```

```
9
10
```

```
9
10
```

- Note in the `increment` function that only `(*ptr)++` gives you the correct result but not `*ptr++;` Why is that?

  > `(*ptr)++` accesses the value that `ptr` points to and increments it by one (which is what we want). `(*ptr++) = *(ptr + 1)` accesses the value of the address following the one `ptr` points at (nothing happens to the value of `value`), since postfix `++` has higher precedence than indirection `*`.

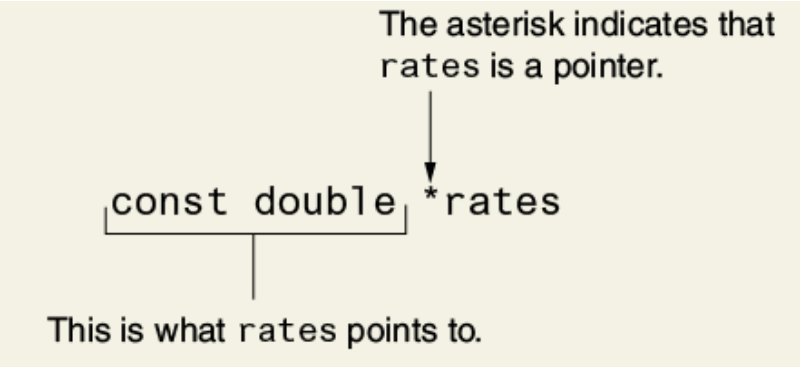
33. Pointers to `const`, `const` pointers, `const` pointers to `const` (skip)

Why would you use these?

1. You use pointers to `const` to protect the data the pointer points to. The pointer can be re-pointed but it cannot change the value it points to. Use-case: read-only access to data, e.g. function parameters.
2. `const` pointers maintain fixed memory addresses. Use-case: hardware-mapped registers or internal buffers.
3. `const` pointers to `const` give full protection for the pointer's value (immutable address) and for the value the pointer points to. Use case: safe in multithreaded or critical code where immutability needs to be guaranteed.

Pointers to constants

- When you pass the address of a `const` item to a pointer, the pointer must be defined as `const`, too.




```
#include <iostream>
#include <iomanip>
using namespace std;

void displayPayRates(const double*, int);

int main()
{
  const int SIZE = 6;
  const double payRates[SIZE] = {18.55, 17.45, 12.85,
                                 14.97, 10.35, 18.89};
  displayPayRates(payRates, SIZE);
  return 0;
}

void displayPayRates(const double *rates, int size)
{
  cout << setprecision(2) << fixed << showpoint;
  for (int count=0; count<size; count++)
    {
      cout << *(rates + count) << endl;
    }
}
```

```
18.55
17.45
12.85
14.97
10.35
18.89
```

```
18.55
17.45
12.85
14.97
10.35
18.89
```

- A pointer to `const` can also receive the address of a non-constant item. When a function does not change the data the parameter points to, you should make the paremeter a pointer to `const`.

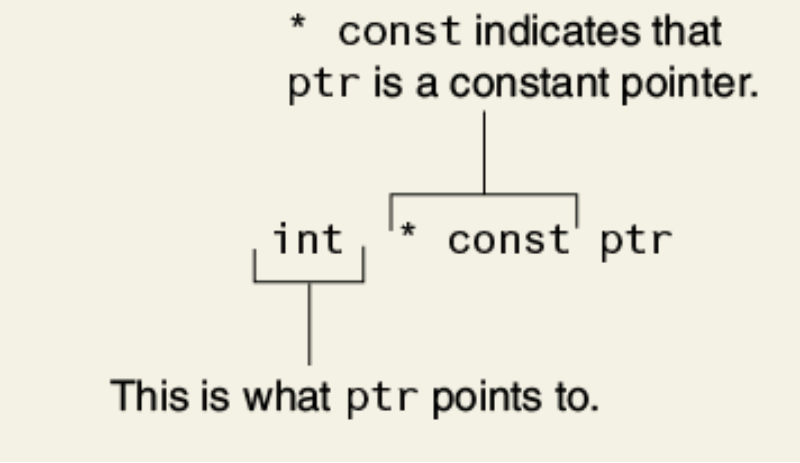
```
#include <iostream>
using namespace std;

void displayValues(const int*, int);

int main()
{
  const int SIZE = 6;
  const int array1[SIZE] = {1,2,3,4,5,6};
  int array2[SIZE] = {2,4,6,8,10,12};

  displayValues(array1, SIZE);
  displayValues(array2, SIZE);

  return 0;
}

void displayValues(const int *nums, int size)
{
  for (int count=0; count < size; count++)
    {
      cout << *(nums + count) << " ";
    }
  cout << endl;
}
```

```
1 2 3 4 5 6
2 4 6 8 10 12
```

Constant pointers

- Differences between a pointer to `const` and a `const` pointer:

    1. A pointer to `const` points towards an immutable item. The data that the pointer points to cannot change the but pointer itself (storing the address of the constant) can change.
    2. A `const` pointer is a pointer that cannot point to anything but the address it has first been initialized with. `const` pointers must be initialized.



- Code examples:

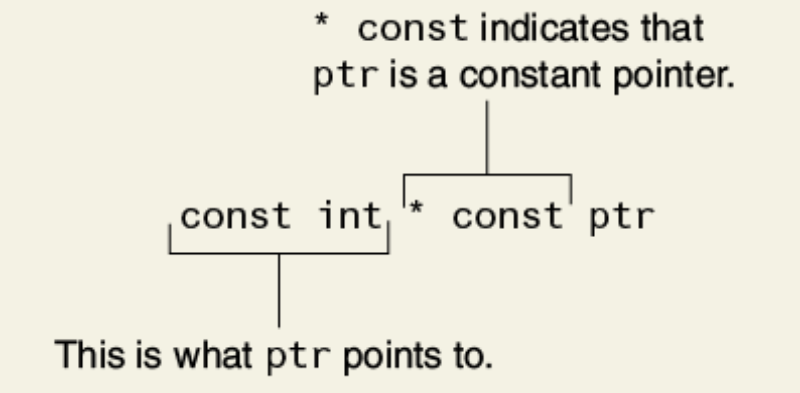
```
int * const ptr; // ILLEGAL: ptr must be initialized

int value = 22;
int * const ptr2 = &value;

int value2 = 44;
ptr2 = &value2  // ILLEGAL: assignment of read-only variable
```

- Here is an example of a function that will not compile since `ptr` is `const`. If you change the statement to `*ptr = 0` it will work.

```
void setToZero(int* const ptr)
{
  ptr = 0; // ILLEGAL - ptr is read-only
}

int main() {return 0;}
```

Constant pointers to constants

- In the example, `ptr` is a `const` pointer to a `const` integer.




```
int value = 22;
const int * const ptr = &value;
```

- The following version of `displayValues` can be called with different arguments but the function itself cannot change what `numbers` point to, and it cannot use `numbers` to change the contents of an argument.

```
void displayValue(const int * const numbers, int size)
{
  for (int count=0; count < size; count++)
    {
      cout << *(numbers + count) << " ";
    }
  cout << endl;
}
int main()
{
```

```
  const int SIZE = 5;
  const int num1[SIZE] = {1,2,3,4,5};
  int num2[SIZE] = {6,7,8,9,10};

  displayValue(num1, SIZE);
  displayValue(num2, SIZE);

  return 0;
}
```

```
1 2 3 4 5
6 7 8 9 10
```

Review questions

1. Give an example of the proper way to call the following function:

```
void makeNegative(int *val)
{
  if (*val > 0) *val = -(*val);
}
```

```
#include <iostream>
using namespace std;
void makeNegative(int *val)
{
  if (*val > 0) *val = -(*val);
}
int main()
{
  int foo = 5;
  makeNegative(&foo);
  cout << foo;
  return 0;
}
```

```
-5
```

2. Suppose a function parameter is declared as follows. According to the declaration, what is constant?

```
const int *ptr;
```

- [ ] The `ptr` pointer
- [X] Any value that `ptr` points to
- [ ] Both the `ptr` pointer and any value that `ptr` points to

3. Suppose a function parameter is declared as follows. According to the declaration, what is constant?

```
int * const ptr;
```

- [X] The `ptr` pointer
- [ ] Any value that `ptr` points to

- [ ] Both the `ptr` pointer and any value that `ptr` points to

4. Look at the following array definition:

```
const int numbers[SIZE] = {18,17,12,14};
```

Suppose that want to pass the array to the function `processArray` in the following manner:

```
processArray(numbers, SIZE);
```

Which of the following function headers is the correct one for the `processArray` function?

1. [X] `void processArray(const int *arr, int size)`
2. [X] `void processArray(int const *arr, int size)`
3. [ ] `void processArray(const int const *arr, int size)`
4. [X] `void processArray(const int * const arr, int size)`

- Explanation:

  In (1,2,3) the parameters declare a pointer to a `const int`: You cannot change `*arr` but you can change `arr`. One of the `const` in (3) is redundant and will not compile. In (4), which is also valid, the pointer is also restricted to be `const`.

- Let's test this:

```
#include <iostream>
using namespace std;

const int SIZE = 4;
const int numbers[SIZE] = {18, 17, 12, 14};

void processArray1(const int *arr, int size) {
  std::cout << "const int *arr\n";
  for (int i = 0; i < size; ++i)
    std::cout << arr[i] << " ";
  std::cout << "\n";
}

void processArray2(int const *arr, int size) {
  std::cout << "int const *arr\n";
  for (int i = 0; i < size; ++i)
    std::cout << arr[i] << " ";
  std::cout << "\n";
}

// void processArray3(const int const *arr, int size) {
//   std::cout << "const int const *arr (redundant)\n";
//   for (int i = 0; i < size; ++i)
//     std::cout << arr[i] << " ";
//   std::cout << "\n";
// }

void processArray4(const int * const arr, int size) {
  std::cout << "const int * const arr\n";
  for (int i = 0; i < size; ++i)
    std::cout << arr[i] << " ";
  std::cout << "\n";
```

```
}

int main() {
  processArray1(numbers, SIZE);
  processArray2(numbers, SIZE);
  processArray4(numbers, SIZE);
}
```

```
const int *arr
18 17 12 14
int const *arr
18 17 12 14
const int * const arr
18 17 12 14
```

34. 9.8 Dynamic memory allocation

- Variables may be created and destroyed while a program is running.
- If you know how many variables you need during execution, you can define them up front:
    1. To calculate the area of a rectangle: `length`, `width`, `area`.
    2. To compute payroll for 30 employees: an array `payRoll[30]`.
- When you don't know that, you need to allocate memory on the fly, during program execution:
    1. To store a list of names from a file if you don't know the number of names in the list.
    2. To run an image processing application where you need to store pixel data based on the resolution of images loaded at runtime.
- Most realtime data problems are of the second sort. They require **dynamic memory allocation**, and you need pointers for that.

- How is this done?

    The program, while running, asks the computer to set aside a chunk of unused memory ("free store", or "heap memory") large enough to hold a variable of a specific type and provides its starting address. To access this memory, the address is needed so a pointer is required to use these bytes.

- In C++, heap memory is requested with the `new` operator: In the following code, the pointer `iptr` holds the starting address to the memory required for one new `int` variable not known at compile time.

```
int *iptr = nullptr;
iptr = new int;
```

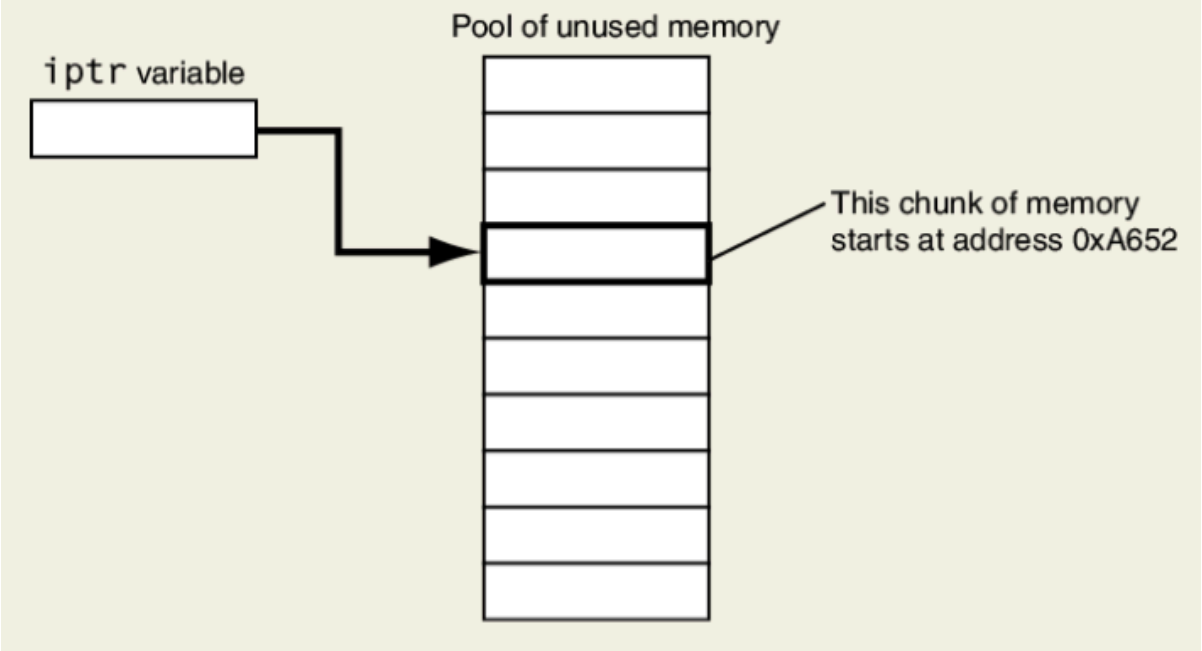


Figure 7: iptr points to a chunk of dynamically allocated memory (Fig 9-11).

- To operate on the `new` variable, the dereferenced pointer `*iptr` is used, for example:

```
int *iptr = nullptr;
iptr = new int;
int total = 100;

cout << *iptr << endl;
cin >> *iptr;
cout << (total += *iptr);
```

- Input

```
cd ../data
echo "1000" > dynamic.txt
cat dynamic.txt
```

```
1000
```

- This mechanism is not typically used for single variables. A practical use is the creation of arrays, containers like `vector`, instances of a `class`, or container adapters like a `stack` though for container adapters, the underlying container manages the memory.

35. Difference to C's `malloc`

Feature	`new` (C++)	`malloc` (C)
Returns	typed pointer	`void*` (must cast)

Feature	`new` (C++)	`malloc` (C)
Initializes memory	Yes (calls constructor)	No (undefined values)
Syntax	operator	function
Deallocation	`delete` / `delete[]`	`free()`
Can allocate objects	Yes (constructors run)	No (just raw memory)

The `new` operator is type-safe and object-aware. `malloc` is raw and low-level. In C++, prefer `new`. Or you can use RAII with containers[2], or smart pointers.

36. Dynamically allocating an array

- Reserving memory at compile time with a `const SIZE` variable works great as long as you know the length of an array:

```
const int SIZE = 4;
int testScores[SIZE] = {0};
```

- What if we want to ask a user for the number of test scores and we want to store them in an array (without knowing the user's answer)?
- Using the test scores example:

```
// Create a pointer to hold the starter address
int *testScores;

// Create a variable that will hold the size when it is known
int size;

// Get the number of test scores during execution
cin >> size;

// Create the pointer to the array of `size` elements
testScores = new int[size];

// Use testScores to store test scores
for (int i=0;i<size;i++)
  {
    cout << "\nEnter score number " << (i + 1) << ": ";
    cin >> testScores[i];
    cout << testScores[i];
  }

// Delete the array (using the pointer)
delete [] testScores;
testScores = nullptr;
```

- Failure to deallocate dynamic memory can cause a program to have a **memory leak**. If memory is dynamically allocated in a function without being deleted, the access pointer is lost and the memory can no longer be accessed and just sits there while the program is live.
- Input

```
cd ../data
echo "3" > testScores.txt # size
echo "95 78 88" >> testScores.txt
cat testScores.txt
```

```
3
95 78 88
```

37. PRACTICE Dynamically Allocated String Array in C++

- Using the starter code below, write a program that asks for the number of participants of an event and reads in their names.
  1. Define a pointer-to-string `names`.
  2. Define an integer variable `count` for the number of participants.
  3. Ask for the number of participants and input `count`.
  4. Output number to check.
  5. Dynamically allocate `new string[count]` memory for `names` (this reserves a `string` array of `count` names.
  6. Loop over `names` and ask for one name after the next (totalling `count`)
  7. Output each name to check.
  8. `delete` the `string` array.
  9. Assign the `nullptr` to the `names` pointer.
  10. Run the program with the names "Alice", "Bob", and "Carol".

- Sample output:

```
Enter the number of participants: 3
Enter name of participant: Alice
Enter name of participant: Bob
Enter name of participant: Carol
```

- Starter code: https://tinyurl.com/dynamically-allocated

```
// This program asks the user for the number and the names of
// participants, stores the names in an array, and outputs them.
// header files

// namespace directive

// main program
int main()
{
  // Define a pointer-to-string `names`.

  // Define an integer variable `count` for the number of participants.

  // Ask for the number of participants and input `count`.

  // Output count to check

  // Dynamically allocate `new` `string[count]` memory for `names`.

  // Loop over `names` and ask for one name after the next.

  // Output the name after each input to check.
```

```
  // delete the string array.

  // Assign the nullptr to the `names` pointer.

  return 0;
}
```

- Solution:

```
// This program asks the user for the number and the names of
// participants, stores the names in an array, and outputs them.
#include <iostream>
#include <string>
using namespace std;

int main()
{
  // Define a pointer-to-string `names`.
  string *names;
  // Define an integer variable `count` for the number of participants.
  int count;
  // Ask for the number of participants and input `count`.
  cout << "Enter the number of participants: ";
  cin >> count;
  cout << count;
  // Dynamically allocate `new` `string[count]` memory for `names`.
  names = new string[count];
  // Loop over `names` and ask for one name after the next.
  for (int i=0;i<count;i++)
    {
      cout << "\nEnter name of participant: ";
      cin >> names[i];
      // Output the name after each input to check.
      cout << names[i];
    }
  // delete the string array.
  delete [] names;
  // Assign the nullptr to the `names` pointer.
  names = nullptr;

  return 0;
}
```

```
Enter the number of participants: 3
Enter name of participant: Alice
Enter name of participant: Bob
Enter name of participant: Carol
```

- Input file

```
cd ../data
echo "4 Alice Bob Carol Joe" > dynString.txt
cat dynString.txt
```

```
4 Alice Bob Carol Joe
```

```
string *names;
int count;

cin >> count;

names = new string[count];

for (int i = 0; i < count; i++)
  {
    cout << "\nEnter name #" << (i + 1) << ": ";
    cin >> names[i];
    cout << names[i];
  }

delete[] names;
names = nullptr;
```

- Always free memory with `delete[]` to avoid memory leaks.
- Input file:

```
cd ../data
echo "3" > participants.txt # number of names
echo "Alice Bob Carol" >> participants.txt
cat participants.txt
```

```
3
Alice Bob Carol
```

38. Review questions

1. Assume `bp` is a pointer to an `bool`. Write a statement that will dynamically allocate a Boolean variable and store its address in `bp`. Write another statement that will free the memory allocated.

```
bool *bp = nullptr;
bp = new bool;
delete bp;
bp = nullptr;
```

2. Assume `ip` is a pointer to an `int`. Write a statement that will dynamically allocate an array of 500 integers and store it in `ip`. Write another statement that will free the memory again.

```
int *ip = nullptr;
ip = new int[500];
delete [] ip;
ip = nullptr;
```

3. What are the five steps to dynamically allocate and de-allocate memory for an array of double precision floating-point values whose total number is not known?

```
// Declare a pointer-to-double with the array's name
double *arr = nullptr;
// Get the size of the array
```

```
int size;
cin >> size;
// Allocate the array with new
arr = new double[size];
// De-allocate the memory
delete [] arr;
// Reset the pointer
arr = nullptr;
```

39. 9.9 Returning pointers from functions (skip)

- Functions can return pointers. This is useful in many situations: returning a pointer gives access to array or class data, supports chaining operations with linked lists or trees, and avoids copying large arrays.
- However, for this to work, you must be sure that the item referenced by the pointer still exists.

- Example: What does the following function do?

```
char *findNull(char *str) {
  char *ptr = str;
  while (*ptr != '\0')
    ptr++;
  return ptr;
}
```

Answer:

> The function takes a character pointer (start address of a string) as argument, steps through the string and returns the address of the null terminator, i.e. one past the last valid character.

- See it in action, with some subtle changes: Since `findNull` does not actually change anything, it can `return` a `const char*`, and the argument can also be `const`.

```
#include <iostream>
using namespace std;
const char *findNull(const char *str);
int main()
{
  const char *str = "Hello";
  cout << "Length = " << findNull(str)-str << endl;
  return 0;
}
const char *findNull(const char *ptr)
{
  while (*ptr != '\0')
    {
      ptr++;
    }
  return ptr;
}
```

```
Length = 5
```

40. Why to `return` pointers from functions (skip)

- You should return a pointer from a function only if it is
    1. A pointer to an item that was passed into the function as an argument,
    2. A pointer to a dynamically allocated chunk of memory.

- Example: What's wrong with the following function? How can you fix it?

```
#include <iostream>
using namespace std;

string *getFullName()
{
  string fullName[3];

  cout << "Enter your first name: ";
  getline(cin, fullName[0]);

  cout << "Enter your middle name: ";
  getline(cin, fullName[1]);

  cout << "Enter your last name: ";
  getline(cin, fullName[2]);

  return fullName;
}

int main() {return 0;}
```

  Answer:

  > `getFullName` returns value of local variable - a pointer to an array that no longer exists when returning from the function.

- Fix 1)

```
#include <iostream>
using namespace std;

string *getFullName(string fullName[])
{
  cout << "Enter your first name: ";
  getline(cin, fullName[0]);

  cout << "Enter your middle name: ";
  getline(cin, fullName[1]);

  cout << "Enter your last name: ";
  getline(cin, fullName[2]);

  return fullName;
}

int main() {return 0;}
```

- Fix 2)

```
#include <iostream>
using namespace std;

string *getFullName()
```

```
{
  string *fullName = new string[3];

  cout << "Enter your first name: ";
  getline(cin, fullName[0]);

  cout << "Enter your middle name: ";
  getline(cin, fullName[1]);

  cout << "Enter your last name: ";
  getline(cin, fullName[2]);

  return fullName;
}

int main() {
  string *fullName = getFullName();
  delete [] fullName;
  return 0;}
```

41. Program: Getting a pointer to an array of random numbers

- This program uses a function, `getRandomNumbers`, to get a pointer to an array of random numbers. The function accepts an integer argument that is the number of random numbers in the array. The function dynamically allocates an array, populates the array with random values, and returns a pointer to the array.

```
#include <iostream>
#include <random>   // for the randInt function
using namespace std;

// Function prototype
int *getRandomNumbers(int);

int main()
{
  int* numbers; // Pointer pointing to the random number array

  // Get an array of five random numbers.
  numbers = getRandomNumbers(5);

  // Display the numbers.
  for (int count = 0; count < 5; count++)
    cout << numbers[count] << " ";

  // Free the memory.
  delete [] numbers;
  numbers = nullptr;

  return 0;
}

// The getRandomNumbers function returns a pointer to an array of
// random integers. The num parameter is the number of numbers
// requested.
int *getRandomNumbers(int num)
{
  const int MIN = 0;  // min random number
  const int MAX = 100;  // max random number
  int *arr = nullptr; // array to hold the numbers

  // Random number engine
```

```
  random_device engine;
  uniform_int_distribution<int> randInt(MIN,MAX);

  // Return null is num is zero or negative
  if (num <= 0)
    return nullptr;

  // Dynamically allocated array
  arr = new int[num];

  // Populate the array with random numbers
  for (int count = 0; count < num; count++)
    arr[count] = randInt(engine);

  // Return a pointer to the array
  return arr;
}
```

```
16 55 29 18 36
```

42. TODO Program: Duplicating an array

p. 547 `duplicateArray`

> Suppose you are developing a program that works with arrays of integers, and you find that you frequently need to duplicate the arrays. Rather than rewriting the array-duplicating code each time you need it, you decide to write a function that accepts an array and its size as arguments, creates a new array that is a copy of the argument array, and returns a pointer to the new array. The function will work as follows:

43. Review questions

1. Does the following function correctly or incorrectly return a pointer?

```
int *getValues()
{
  int *ptr = new int[3];
  ptr[0] = 0;
  ptr[1] = 0;
  ptr[2] = 0;

  return ptr;
}
```

2. Does the following function correctly or incorrectly return a pointer?

```
int *getValues()
{
  int values[3];
  values[0] = 0;
  values[1] = 0;
  values[2] = 0;

  return values;
}
```

44. 9.10 Using smart pointers [opt]

- C++11 introduced **smart pointers** that work like pointers but will automatically delete dynamically allocated memory that is no longer used so that **memory leaks** are avoided, and you no longer have to use `new` and `delete`, which are abstracted away.
- To use them you need to `#include <memory>`.
- Three types of smart pointers are provided ("memory" = dynamically allocated memory"):
  1. `unique_ptr`: This pointer owns a piece memory. No two `unique_ptr` can point to the same memory. When it goes out of scope, it deletes the memory it points at.
  2. `shared_ptr`: This pointer can share ownership of memory. Multiple pointers can point to the same memory. It is deallocated when the last of these pointers is destroyed.
  3. `weak_ptr`: This pointer does not own the memory it points to, and cannot be used to access the memory's content. It is used when the memory pointed to by a `shared_ptr` must be referenced without increasing the number of these pointers that own it.

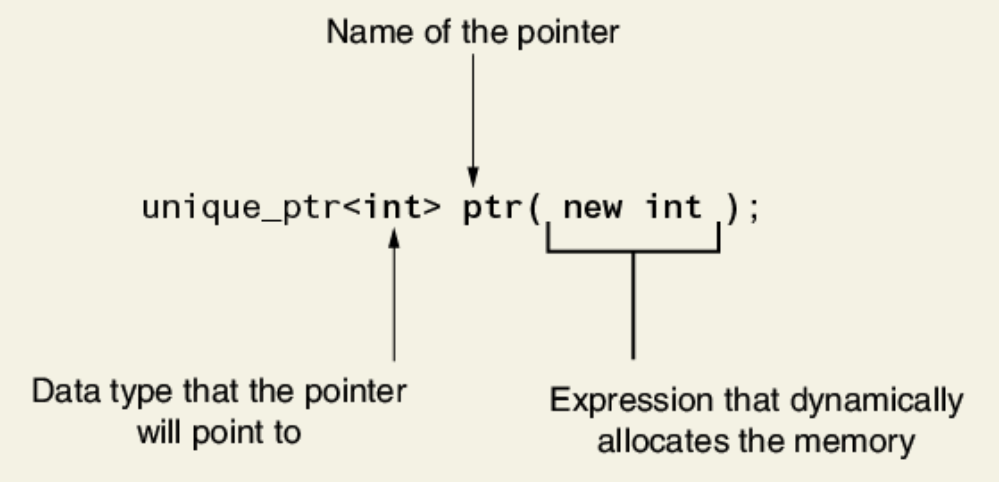
- Summary:

Pointer	Ownership	Notes
`unique_ptr`	Exclusive	Move-only, no copy
`shared_ptr`	Shared	Ref-counted, shared ownership
`weak_ptr`	Observer	Non-owning, no ref-count

There was a fourth smart pointer `auto_ptr`, but i[3]t is **deprecated** in C++11 and was removed in C++17.

45. The `unique_ptr`

- Once you've defined the `unique_ptr` you can use it like a regular pointer.



- Example program: A `unique_ptr` points to a dynamically allocated array of integers.

```
#include <iostream>
#include <memory>
```

```
using namespace std;

int main()
{
  int max; // Max size of array

  // Get the number of values to sore.
  cout << "How many numbers do you want to enter? ";
  cin >> max;
  cout << max; // output check

  // Define a smart pointer to an array of ints
  unique_ptr<int[]> ptr(new int[max]);

  // Get (and print) the values for the array.
  for (int index = 0; index < max; index++)
    {
      cout << "\nEnter an integer number: ";
      cin >> ptr[index];
      cout << ptr[index] << " <RET>"; // output check
    }

  // Display the values in the array.
  cout << "\nHere are the values you entered: ";
  for (int index = 0; index < max; index++)
    {
      cout << ptr[index] << " ";
    }

  return 0;
}
```

- You can define a pointer and later assign it a value:

```
unique_ptr<int> ptr = nullptr;
ptr = unique_ptr<int> (new int);
```

- Smart pointers do **not** support pointer arithmetic:

```
unique_ptr<int[]> ptr(new int[5]);
ptr++; // NOT VALID - operator not declared for smart pointers
```

- Input:

```
cd ../data/ echo "5 1 2 3 4 5" > unique_ptr.txt cat unique_ptr.txt
```

46. TODO 9.11 Case study

47. Glossary

Term	Definition
Address	Unique memory location, 64-bit on modern systems
Address-of (&)	Operator to get a variable's address (&x)

Term	Definition
Dereference (*)	Access value at pointer's address (`*p`)
Pointer	Variable holding a memory address (`int* p`)
Sizeof	Size in bytes of a variable/type (`sizeof(x)`)
Byte	8 bits; smallest addressable memory unit
Hexadecimal	Base-16 format, used to show addresses (`0x...`)
Dynamic Memory	Allocated at runtime using pointers
Reference Variable	Alias for another variable (`int& ref = x`)
String Literal	Constant string stored in memory (`"Hello"`)
Array Decay	Arrays become pointers when passed to functions
Traversal	Moving through elements via loops/pointers
Pointers vs Refs	Pointers can be reassigned; refs = fixed alias
Pointer Use Cases	For dynamic memory, arrays, and object access
RAM	Memory where data & code are stored temporarily
Stack	RAM area for function calls and local variables
Heap	RAM area for memory that is allocated at runtime
`(void *)`	Generic pointer type (for casting)
`ptrdiff_t`	Type for pointer (address) differences
memory leak	Memory allocated dynamically but not deleted
`unique_ptr`	Exclusive ownership, move-only, no copy
`shared_ptr`	Shared ownership, ref-counted, shared ownership
`weak_ptr`	Observer ownership, non-owning, no ref-count

Footnotes:

[1] These statements use the ternary operator (EXPR) ? (TRUE) : (FALSE)

[2] RAII = Resources Acquisition Is Initialization: memory, files and sockets in C++ are tied to the object life time. Resources are acquired in a constructor, released in a destructor, so that there is no need for an explicit `delete` or `free` call.

[3] "Deprecated in C++11" means that its use is discouraged, and "removed in C++17" means you can no longer use it, if the compiler supports this C++ version (or is made to with a `-std` option).

Author: Marcus Birkenkrahe
Created: 2026-03-21 Sat 16:59